# HashVault

**A Secure Web Platform for Hash Generation, Verification, and Integrity Monitoring**

**Vercel link: https://hashvault-youssefhamada2428-coder.vercel.app?_vercel_share=NcEmCECv8M9fjH3ARoqKVAESKewVQiAH**

**GitHub repository: https://github.com/youssefhamada2428-coder/hashvault**

## MADE BY

Mostafa Yasser Abdelfattah Ewis -24102568

Yousef Hamada Yousef Isaa-24102779

## Under supervisor

DR. Moataz Samy
DR. Marwa Suliaman

## Course

Web technology
Mental Wealth

## Contents

# HashVault

**A Secure Web Platform for Hash Generation, Verification, and Integrity Monitoring**

## 1) Project Overview

**HashVault** is a full-stack web-based integrity verification platform designed to help users generate, compare, and manage cryptographic hashes for both files and plain text. It supports widely used hashing algorithms including **MD5, SHA-1, SHA-256, and SHA-512**, providing flexibility for different security and performance needs.

The platform goes beyond simple hash generation by offering:

- Drag-and-drop file hashing
- Real-time text hashing
- Hash comparison engine
- Historical tracking and storage via Supabase
- Interactive dashboards for monitoring hash activity
- Admin automation tools for managing users and processes

HashVault transforms basic hashing into a **complete integrity management system**, suitable for developers, security analysts, and organizations.

## 2) Problem Statement

Ensuring data integrity is critical in modern systems, especially in cybersecurity, software development, and data transmission. However, existing hashing tools present several limitations:

- Lack of persistent history tracking
- No structured way to compare multiple hashes over time
- Minimal or no user interface (CLI-heavy tools)
- Absence of dashboards or monitoring features
- No centralized admin control for teams or organizations

As a result, users struggle to:

- Verify file authenticity efficiently
- Track changes across versions
- Manage hashing operations at scale

**HashVault addresses these gaps** by providing a centralized, user-friendly, and scalable platform for hash-based integrity verification.

## 3) Objectives

HashVault aims to:

- Provide a **secure and intuitive interface** for generating hashes
- Enable **accurate comparison** between files and text inputs
- Maintain a **history of hashing activities** for audit and tracking
- Offer **dashboard insights** for monitoring usage and trends
- Support **admin-level controls** for managing users and workflows
- Ensure **scalability and performance** using modern web technologies

## 4) Core Features

**Hash Generation**

- Generate hashes for:
  - Plain text input
  - Uploaded files
- Supported algorithms:
  - MD5
  - SHA-1
  - SHA-256
  - SHA-512

**Hash Verification & Comparison**

- Compare two hashes (text vs text, file vs file)
- Detect: Exact matches and Mismatches (potential tampering)
- Visual feedback for quick interpretation

**History & Logging**

- Store all hash operations in **Supabase database**
- Track: Timestamp, Algorithm used, and Input type (file/text)
- Allow users to revisit and reuse previous hashes

**Dashboard & Monitoring**

- Visualize:
    - Number of hashes generated
    - Algorithm usage distribution
    - Recent activity logs
- Helps in project tracking and auditing

**Admin Automation**

- Manage users and permissions
- Monitor system usage
- Automate cleanup or archival of old records
- Control access to advanced features

**User Experience**

- Clean UI built with modern frontend frameworks (Next.js + Tailwind)
- Drag-and-drop file upload
- Real-time feedback and results
- Responsive design for all devices

# 5) System Architecture (High-Level)

**Frontend**

- Next.js (React-based framework)
- Tailwind CSS for UI

**Backend**

- Supabase (Database + Auth + Storage)

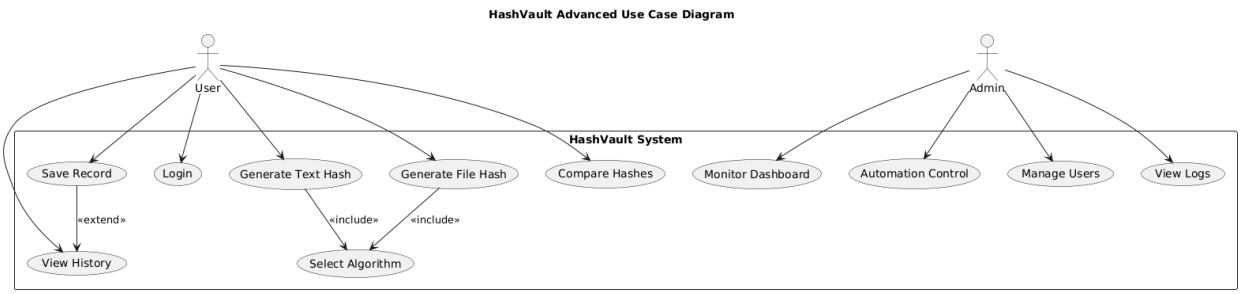
**Core Logic**

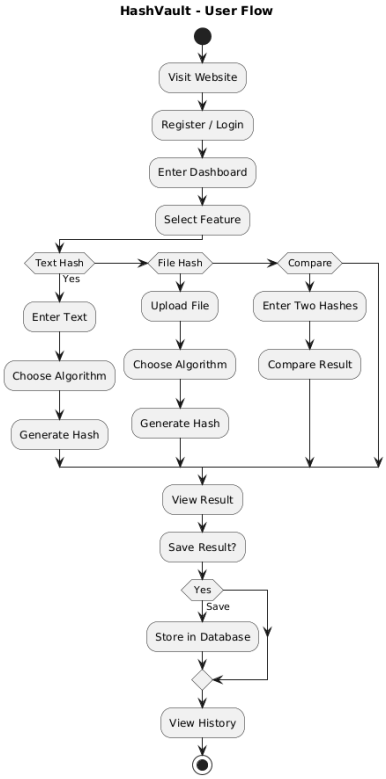
- Hashing handled via: Node.js crypto module

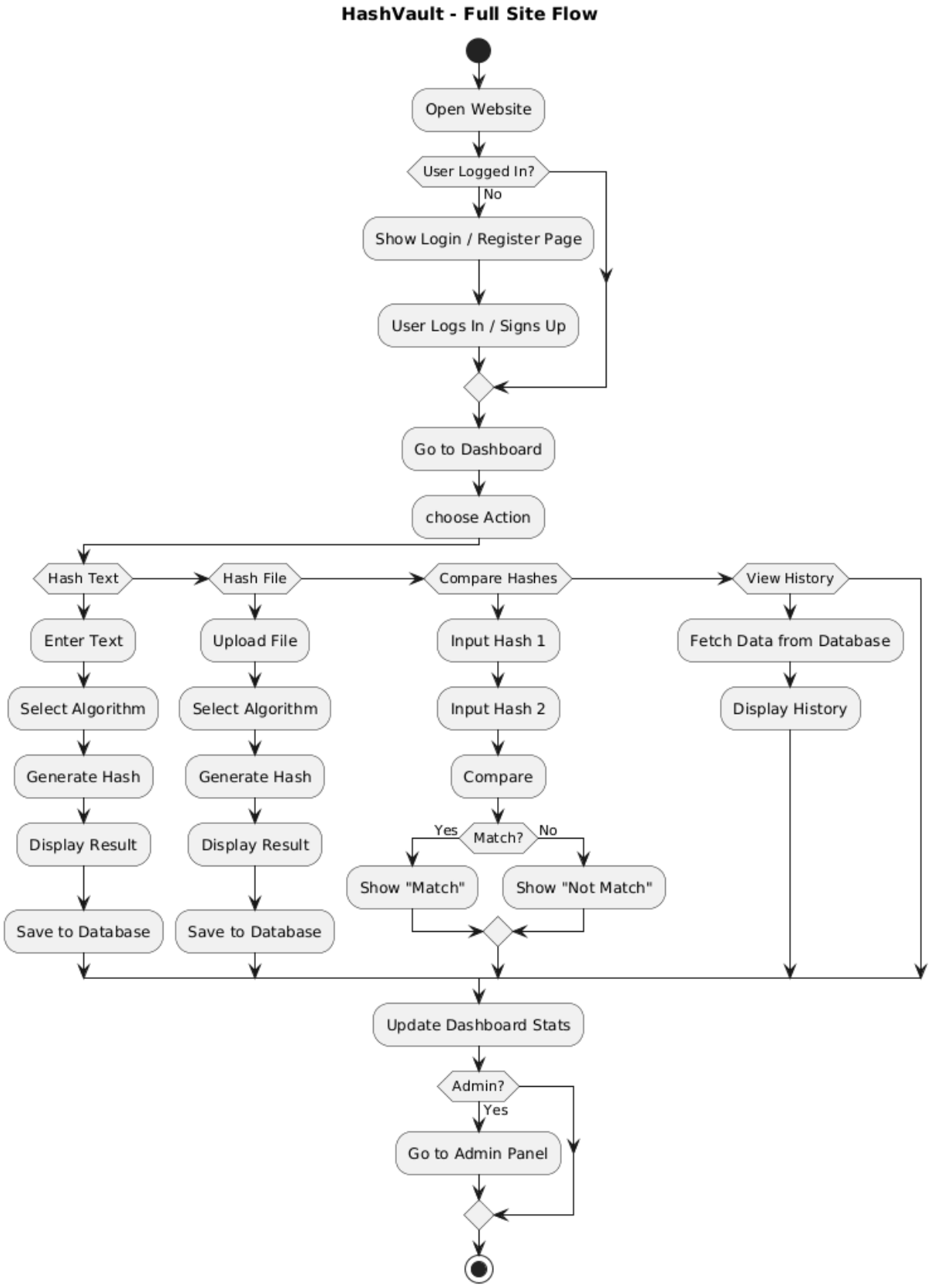
**6) Security Considerations**

- Secure handling of file uploads
- No sensitive file storage unless explicitly required
- Role-based access for admin features
- Input validation to prevent abuse

# 7) Simple UML Diagrams




HashVault - User Flow
Visit Website
Register / Login
Enter Dashboard
Select Feature
Text Hash
File Hash
Compare
Yes
Enter Text
Upload File
Enter Two Hashes
Choose Algorithm
Choose Algorithm
Compare Result
Generate Hash
Generate Hash
View Result
Save Result?
Yes
Save
Store in Database
View History


HashVault - Full Site Flow
Open Website
User Logged In?
No
Show Login / Register Page
User Logs In / Signs Up
Go to Dashboard
choose Action
Hash Text
Hash File
Compare Hashes
View History
Enter Text
Upload File
Input Hash 1
Fetch Data from Database
Select Algorithm
Select Algorithm
Input Hash 2
Display History
Generate Hash
Generate Hash
Compare
Display Result
Display Result
Yes
Match?
No
Show "Match"
Show "Not Match"
Save to Database
Save to Database
Update Dashboard Stats
Admin?
Yes
Go to Admin Panel

## 8) Future Enhancements

- API access for developers
- Bulk file hashing
- Cloud storage integration
- Alerts for hash mismatches
- AI-based anomaly detection
- Browser extension

## 9) Project Value

HashVault is not just a hashing tool—it is a **complete integrity verification ecosystem** that combines usability, security, and scalability. It demonstrates strong skills in:

- Full-stack development
- Cybersecurity fundamentals
- UI/UX design
- Database integration
- Real-world problem solving

## 10) Requirements Scope

**In Scope**

Functional requirements:

- Generate hashes from uploaded files
- Generate hashes from entered text
- Compare two hashes and show match/mismatch
- Save Hash History in Supabase
- Search, view, edit, and delete history items
- Authentication system
- Role-based access control using Supabase RLS

- Admin dashboard with project metrics
- Configuration management screens
- Requirements monitoring and traceability

Non-functional requirements:

- Responsive design
- Secure data handling
- Fast hashing performance
- Clear UX for non-technical users
- Maintainable code structure
- Version control with GitHub

**Out of Scope**

- File sharing between users
- Encryption/decryption
- Malware scanning
- Blockchain-based hashing
- Multi-tenant enterprise billing

## 11) Core Website Pages and Screens

You need **at least 4 CRUD screens**. I recommend these 6:

1. **Home / Hash Generator**
    - drag-and-drop file hashing
    - text hashing input
    - algorithm selector
    - result output

2. **Hash History CRUD**
    - create, read, update, delete saved hash records
    - filters by date, algorithm, file type
    -


3. **Comparison Tool CRUD**
    - save comparison records
    - compare two hashes
    - store verification status
4. **Configuration Management CRUD**
    - manage allowed algorithms
    - export formats
    - retention settings
    - notification preferences
5. **Requirements Monitoring Dashboard**
    - requirement IDs
    - implementation status
    - test coverage
    - traceability links
6. **Admin Analytics Dashboard**
    - KPI cards
    - burndown chart
    - velocity chart
    - usage stats

- error logs

## 12) Technical Architecture

**Frontend**

- **Next.js**
- Routing: App Router
- State: React state + server actions where suitable
- SEO: metadata, clean route structure, semantic HTML
- UI styling: **Stitch-generated design system**

**Backend**

- **Supabase**
- Database tables for:
  - users
  - hash_history
  - comparison_logs
  - configuration_settings
  - requirements
  - sprint_metrics

**Security**

- Supabase Auth
- Row-Level Security
- Protected routes
- Input validation
- Safe file size limits
- Audit logging

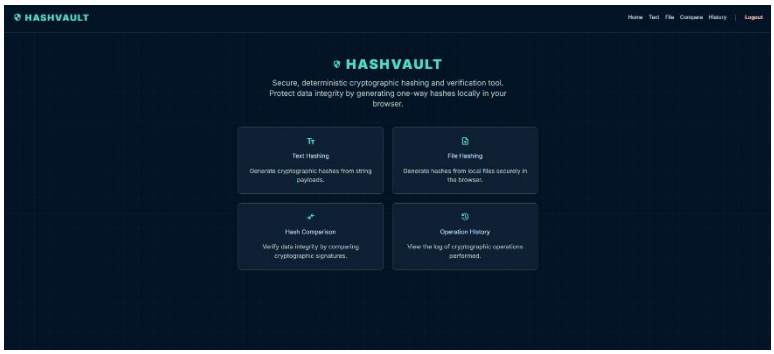
# 13) Suggested Database Tables

- users
- hash_history
- comparison_logs
- configurations
- requirements
- test_cases
- sprints
- task_metrics

# 14) Task Planning

**Phase 1: Planning and Analysis**

- define requirements
- create wireframes
- map modules and screens
- prepare database schema

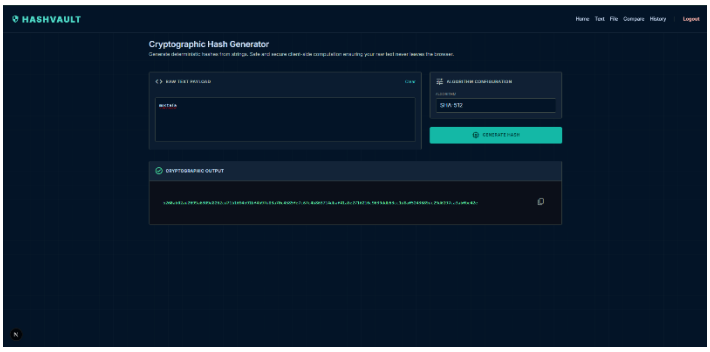



**Phase 2: UI/UX Design**

- create main screens in Stitch
- build responsive layouts
- define colors, typography, buttons, cards, and forms

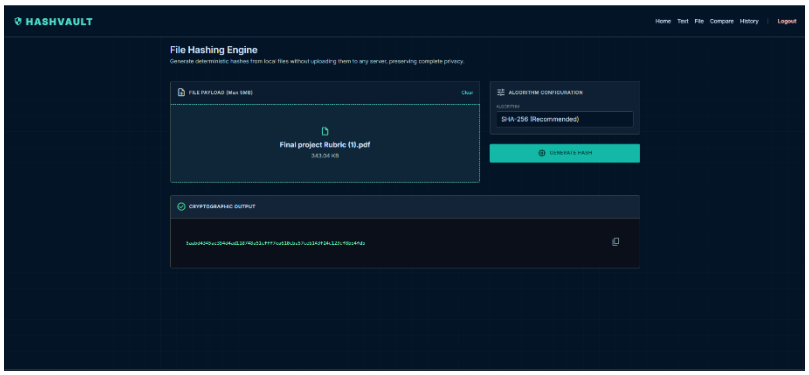
**Phase 3: Frontend Development**

- implement routes
- build hashing forms
- build drag-and-drop area
- build history pages
- build dashboards




**Phase 4: Backend Integration**

- configure Supabase auth
- create tables
- define RLS policies
- connect CRUD actions
- store and retrieve history




**Phase 5: Testing and Optimization**

- unit tests
- integration tests
- validation checks
- responsiveness testing
- security verification

**Phase 6: Deployment and Documentation**

- deploy to Vercel
- add README
- prepare final report
- include diagrams, KPIs, and financial analysis

# 15) Risks Analysis

| Risk | Impact | Probability | Mitigation |
|---|---|---|---|
| Incorrect hash comparison logic | High | Low | unit tests for all algorithms |
| Supabase auth/RLS misconfiguration | High | Medium | test policies carefully |
| Large file upload slowdown | Medium | Medium | file size limits and progress feedback |
| Poor UX on mobile | Medium | Medium | responsive layout testing |
| Scope creep | High | High | freeze requirements early |
| Data loss in history | High | Low | backups and database rules |
| Delayed implementation | Medium | Medium | sprint planning and task tracking |

# 16) Financial Analysis

**Assumptions**

- **Development effort:** 160 hours
    - (Calculation: 2 workers × 40 hours/week × 2 weeks)
- **Estimated value of work per hour:** $15
- **Development cost:** 160 × 15 = $2,400
- **Annual operating cost:** $150
    - Domain, Supabase Pro tier, and minimal maintenance

**Value created by the tool:** * 400 users/month

- Each save 5 minutes
- Average labor value = $25/hour

**Annual Value Calculation**

- **Time saved per month:** * 400 users × 5 minutes = 2,000 minutes
    - 2,000 / 60 = 33.33 hours/month
- **Monthly value:** * 33.33 × $25 = $833.25/month
- **Annual value:** * 833.25 × 12 = $10,000/year

**Net Annual Benefit**

- **Annual value:** $10,000
- **Less operating cost:** $150
- **Net annual benefit** = $9,850

**ROI**

- **ROI** = (Net benefit - development cost) / development cost × 100
- **ROI** = (9850 - 2400) / 2400 × 100
- **ROI** = 310.4%

**Payback Period**

- **Monthly net benefit** = 9850 / 12 = $820.83
- **Payback** = 2400 / 820.83 = 2.92 months

**Financial Conclusion** The project is financially justified as a high-efficiency investment. With two developers delivering the solution in two weeks, the high ROI and sub-3-month payback period demonstrate significant value in automating cryptographic workflows.

# 17) Decision Analysis

**Alternatives**

1. **Static local-only hashing tool**
2. **Next.js + Supabase custom app**
3. **Heavy backend enterprise solution**

**Weighted Decision Matrix**

| Criteria | Weight | Static Tool | Next.js + Supabase | Enterprise Backend |
|---|---|---|---|---|
| User experience | 20% | 3 | 5 | 4 |
| Security | 20% | 1 | 5 | 5 |
| Scalability | 20% | 2 | 5 | 5 |
| Speed of development | 15% | 5 | 4 | 2 |
| Maintainability | 15% | 3 | 5 | 4 |
| University mark fit | 10% | 2 | 5 | 4 |

**Result**

- Static tool: **2.55 / 5**
- Next.js + Supabase: **4.85 / 5**
- Enterprise backend: **4.00 / 5**

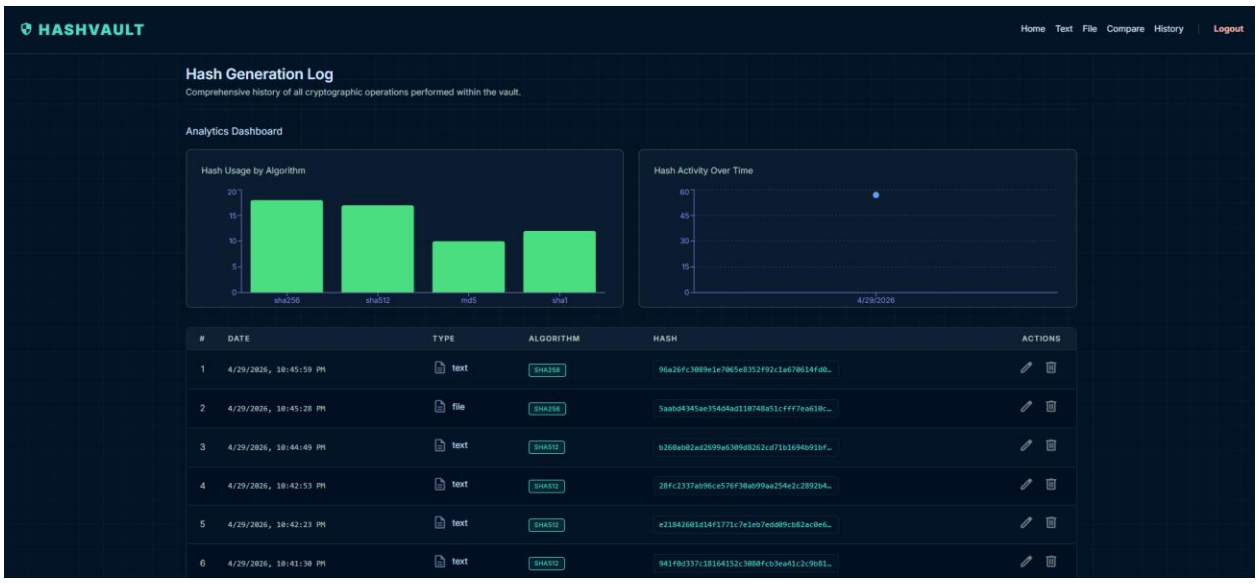
**Decision**

**Choose Next.js + Supabase** because it gives the best balance of usability, security, scalability, and academic value.

# 18) KPIs and Dashboard Metrics

Track these on the dashboard:

- number of hashes generated
- number of file uploads
- number of text hashes
- comparison success rate
- average processing time
- active users
- error rate
- history entries saved
- task completion rate
- sprint velocity

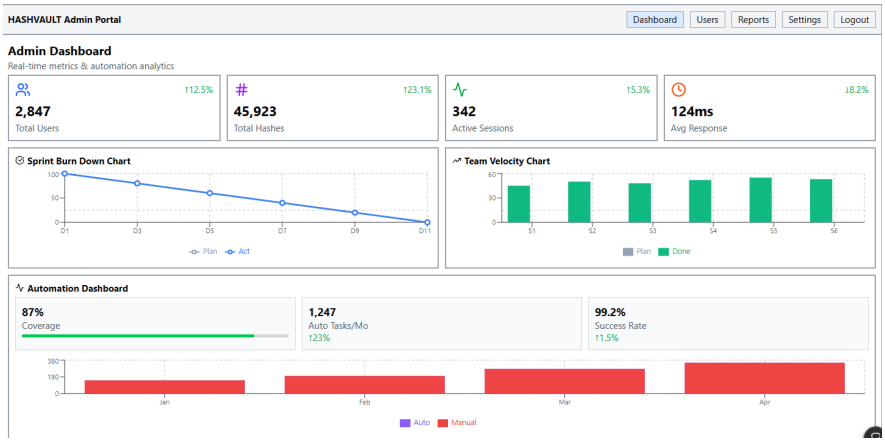


**Burndown Chart**

Tracks remaining project tasks across the sprint.

**Velocity Chart**

Tracks how many story points are completed per sprint.




**Automation on Dashboard**

- auto-update metrics from Supabase
- weekly sprint summaries
- requirement status updates
- test pass/fail counts

# 19) Configuration Management and Version Control

Use:

- **GitHub repository**
- branch strategy:
  - main
  - develop
  - feature branches
- semantic versioning:
  - v1.0.0, v1.1.0
- commit conventions:
  - feat:
  - fix:
  - docs:

  - refactor:
- release notes for each version
- environment variables managed securely

## 20) Requirements Mapping and Monitoring Procedure

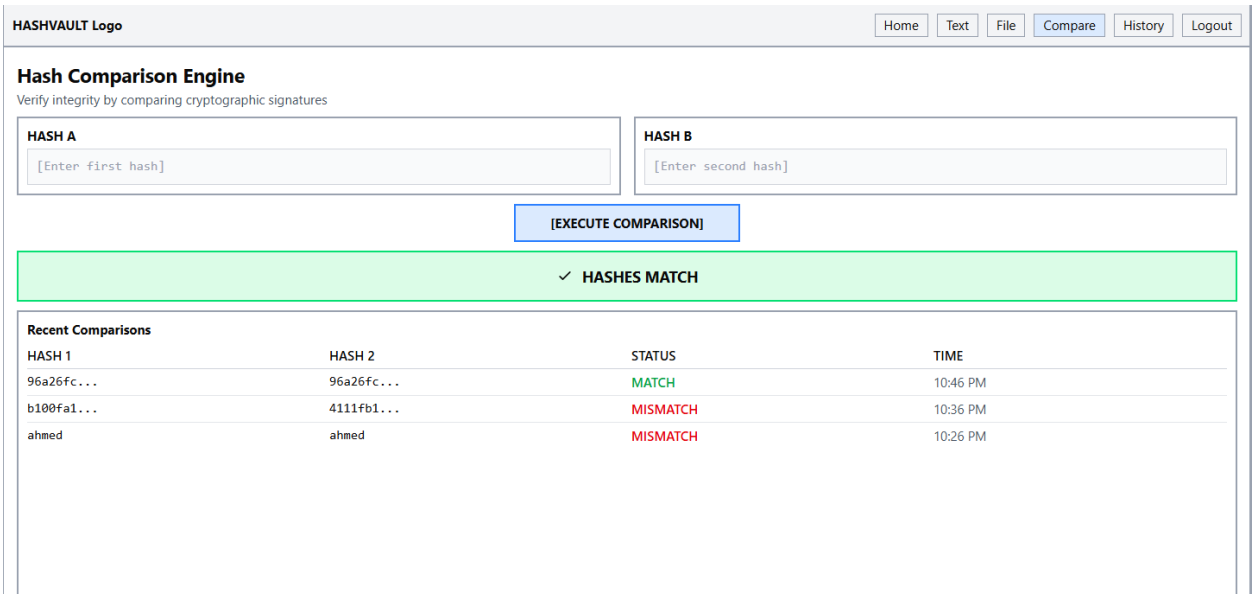
Create a traceability matrix like this:

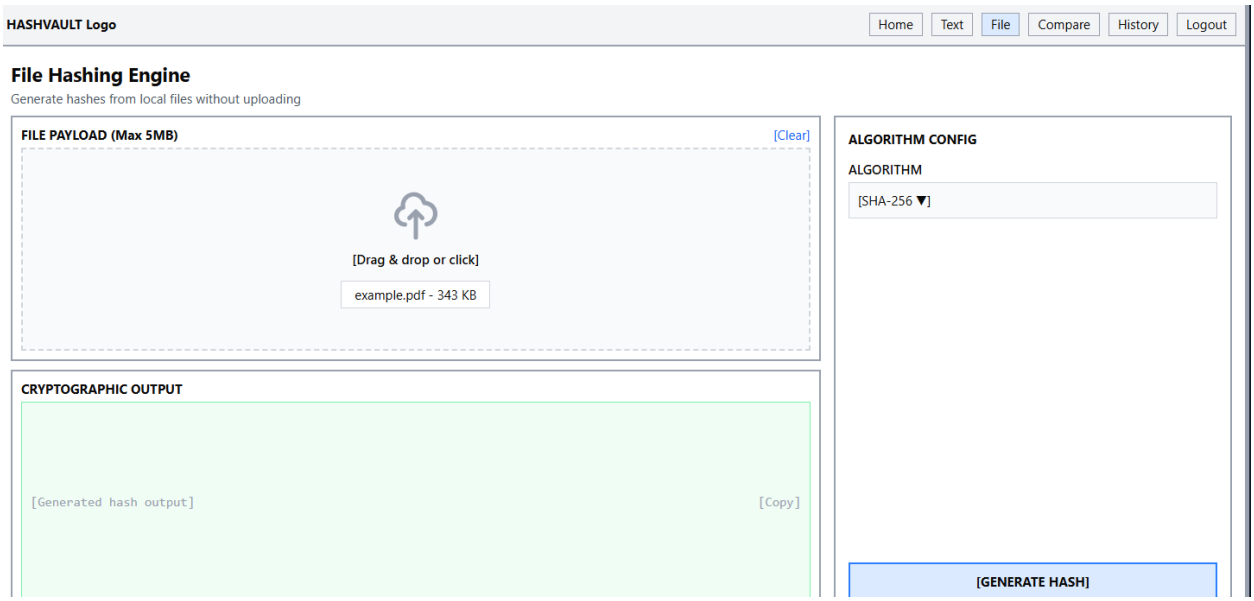
| Req ID | Requirement | Screen | DB Table | Test Case | Status |
|---|---|---|---|---|---|
| 1 | Generate file hash | Home | hash_history | TC-01 | Done |
| 2 | Generate text hash | Home | hash_history | TC-02 | Done |
| 3 | Compare hashes | Compare Tool | comparison_logs | TC-03 | Done |
| 4 | Save history | History | hash_history | TC-04 | Done |
| 5 | Manage settings | Config | configurations | TC-05 | Done |

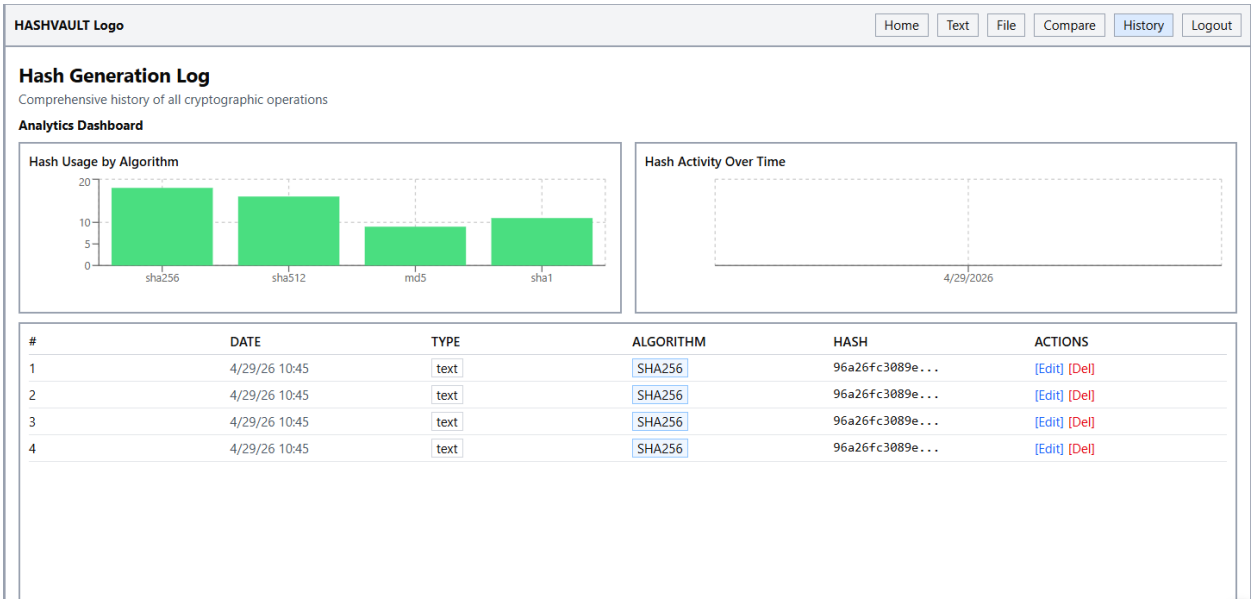
**Monitoring Procedure**

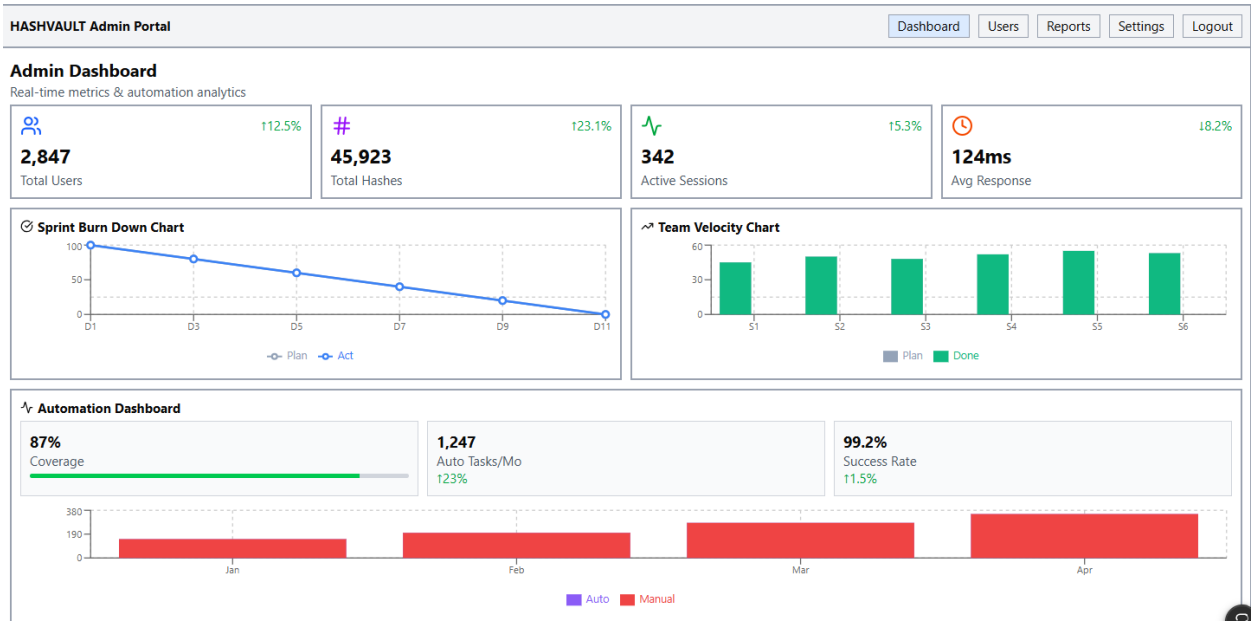
- update requirements weekly
- link each requirement to a screen and test case
- review progress in dashboard
- mark status: planned, in progress, testing, done
- verify all completed items before submission

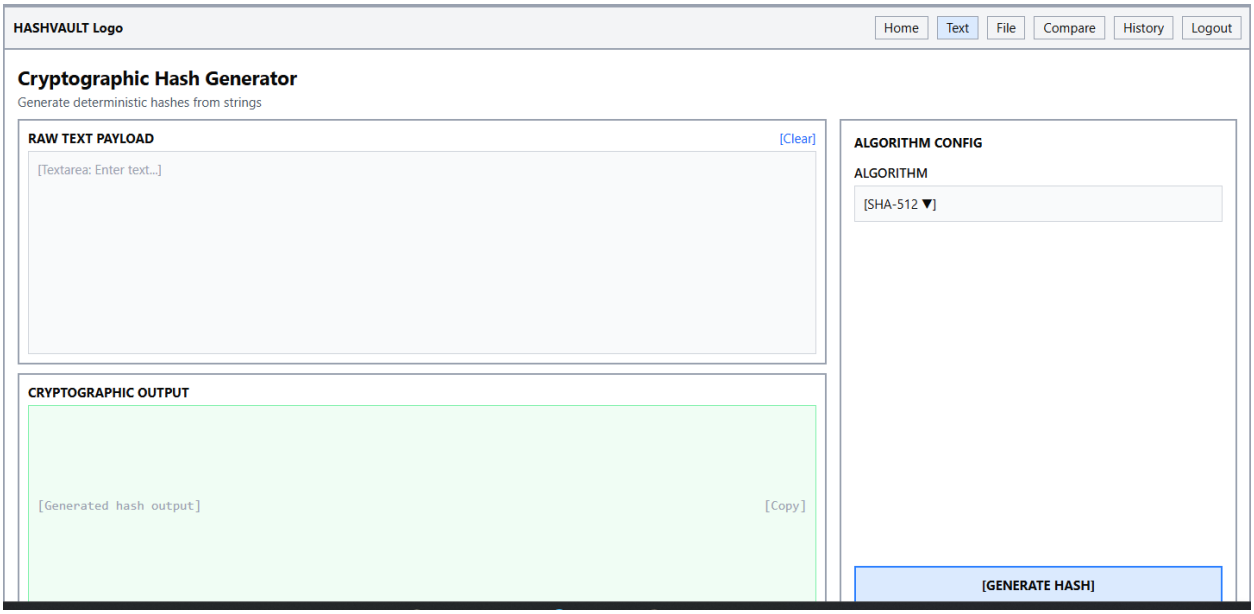
# Wireframes


HASHVAULT Logo
Home
Text
File
Compare
History
Logout
Hash Comparison Engine
Verify integrity by comparing cryptographic signatures
HASH A
[Enter first hash]
HASH B
[Enter second hash]
[EXECUTE COMPARISON]
✓ HASHES MATCH
Recent Comparisons
HASH 1
HASH 2
STATUS
TIME
96a26fc...
96a26fc...
MATCH
10:46 PM
b100fa1...
4111fb1...
MISMATCH
10:36 PM
ahmed
ahmed
MISMATCH
10:26 PM

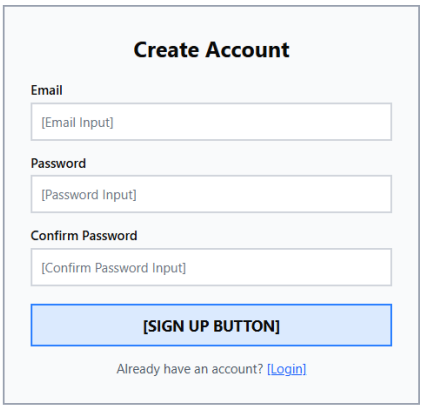

HASHVAULT Logo
Home
Text
File
Compare
History
Logout
File Hashing Engine
Generate hashes from local files without uploading
FILE PAYLOAD (Max 5MB)
[Clear]
[Drag & drop or click]
example.pdf - 343 KB
ALGORITHM CONFIG
ALGORITHM
[SHA-256 ▼]
CRYPTOGRAPHIC OUTPUT
[Generated hash output]
[Copy]
[GENERATE HASH]


HASHVAULT Logo
Home
Text
File
Compare
History
Logout
Hash Generation Log
Comprehensive history of all cryptographic operations
Analytics Dashboard
Hash Usage by Algorithm
20
10
5
0
sha256
sha512
md5
sha1
Hash Activity Over Time
4/29/2026
# DATE TYPE ALGORITHM HASH ACTIONS
1 4/29/26 10:45 text SHA256 96a26fc3089e... [Edit] [Del]
2 4/29/26 10:45 text SHA256 96a26fc3089e... [Edit] [Del]
3 4/29/26 10:45 text SHA256 96a26fc3089e... [Edit] [Del]
4 4/29/26 10:45 text SHA256 96a26fc3089e... [Edit] [Del]

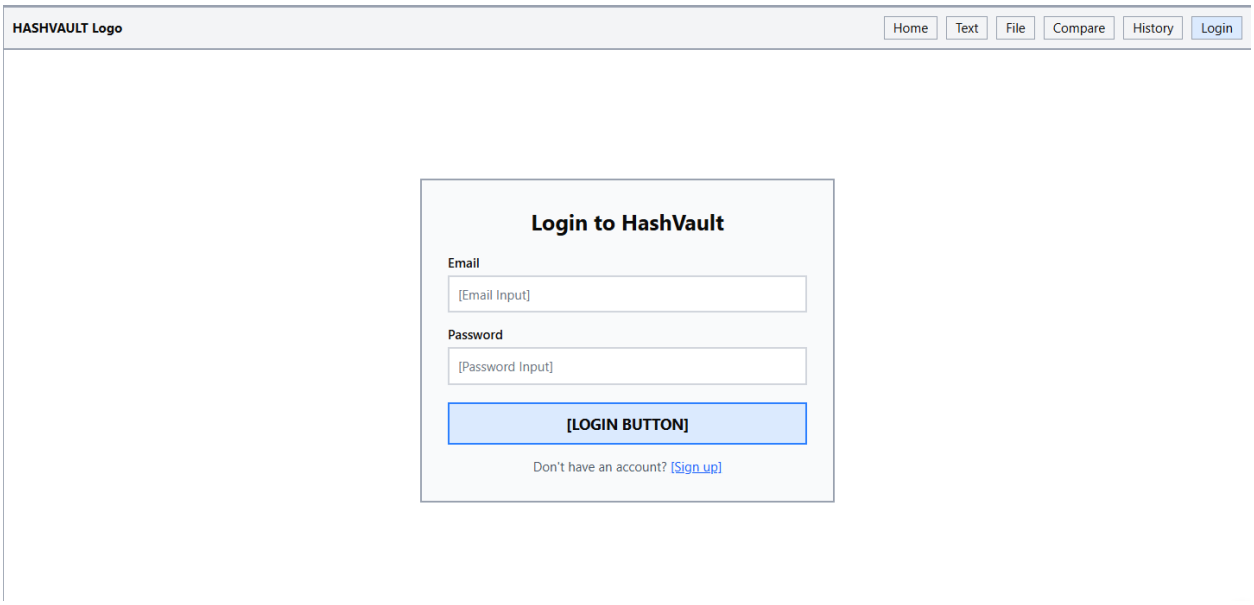

HASHVAULT Admin Portal
Dashboard
Users
Reports
Settings
Logout
Admin Dashboard
Real-time metrics & automation analytics
↑12.5%
2,847
Total Users
↑23.1%
45,923
Total Hashes
↑5.3%
342
Active Sessions
↓8.2%
124ms
Avg Response
Sprint Burn Down Chart
100
50
0
D1
D3
D5
D7
D9
D11
Plan
Act
Team Velocity Chart
60
30
0
S1
S2
S3
S4
S5
S6
Plan
Done
Automation Dashboard
87%
Coverage
1,247
Auto Tasks/Mo
↑23%
99.2%
Success Rate
↑1.5%
380
190
0
Jan
Feb
Mar
Apr
Auto
Manual

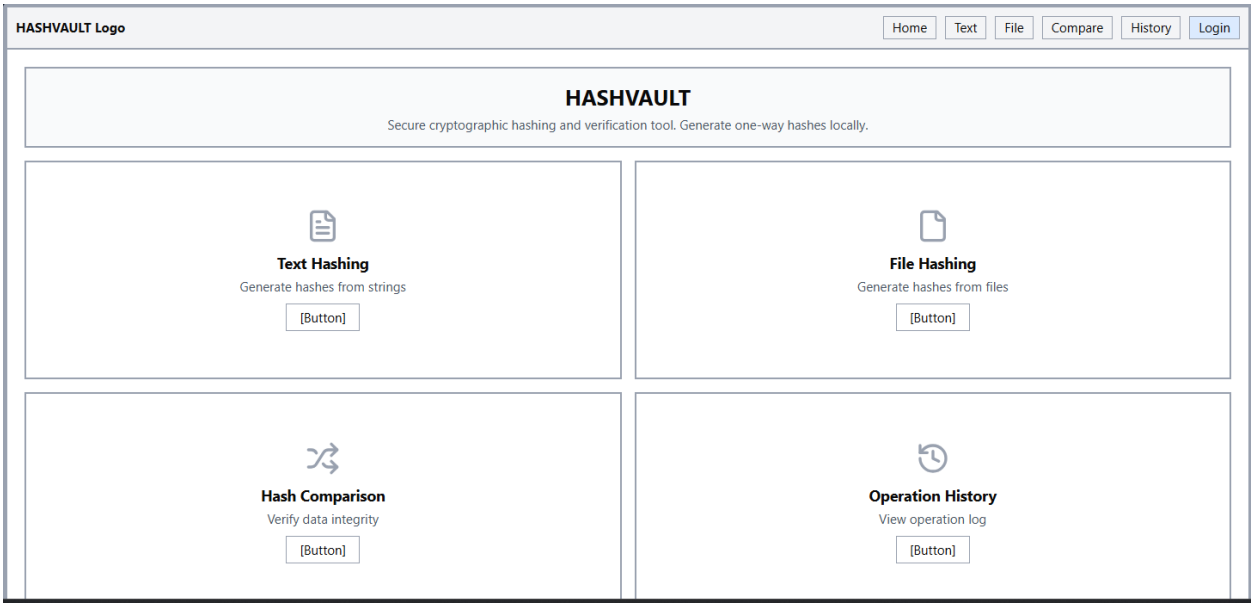

HASHVAULT Logo
Home
Text
File
Compare
History
Logout
Cryptographic Hash Generator
Generate deterministic hashes from strings
RAW TEXT PAYLOAD
[Clear]
[Textarea: Enter text...]
ALGORITHM CONFIG
ALGORITHM
[SHA-512 ▼]
CRYPTOGRAPHIC OUTPUT
[Generated hash output]
[Copy]
[GENERATE HASH]


HASHVAULT Logo
Home
Text
File
Compare
History
Sign Up


Create Account
Email
[Email Input]
Password
[Password Input]
Confirm Password
[Confirm Password Input]
[SIGN UP BUTTON]
Already have an account? [Login]

\


HASHVAULT Logo
Home
Text
File
Compare
History
Login
Login to HashVault
Email
[Email Input]
Password
[Password Input]
[LOGIN BUTTON]
Don't have an account? [Sign up]


HASHVAULT Logo
Home
Text
File
Compare
History
Login
HASHVAULT
Secure cryptographic hashing and verification tool. Generate one-way hashes locally.
Text Hashing
Generate hashes from strings
[Button]
File Hashing
Generate hashes from files
[Button]
Hash Comparison
Verify data integrity
[Button]
Operation History
View operation log
[Button]